

CVE-2024-6387

regreSSHion: unauthenticated remote code execution in OpenSSH sshd

Prepared 25 September 2026 | Classification: Internal | Remediation: upgrade OpenSSH to 9.8p1 or later

A note on the version number. The first fixed release is **OpenSSH 9.8p1** (1 July 2024). 9.9p1 (19 September 2024) and later also contain the fix, so "9.9p1 or later" is a valid target if you build from source. On packaged distributions (Ubuntu, Debian, RHEL and others) you will usually stay on the older version number and receive the fix as a backport, so always verify by package changelog, not by version string alone (see section 5).

1. Executive summary

CVE-2024-6387, named **regreSSHion** by the Qualys Threat Research Unit, is a race condition in the OpenSSH server (sshd). When a client connects and does not finish authenticating within `LoginGraceTime` (default 120 seconds), sshd runs a `SIGALRM` handler that calls functions that are not safe to call from a signal handler. If the timing lines up, this corrupts memory and can lead to code execution as **root**, with no credentials and no user interaction. It is a regression of CVE-2006-5051, a bug fixed in 2006 that was accidentally reintroduced in 2020.

Item	Detail
CVE / name	CVE-2024-6387 / regreSSHion
CVSS 3.1	8.1 High, AV:N/AC:H/PR:N/UI:N/S:U/C:H/I:H/A:H (NVD). The high attack complexity reflects the race condition.
Weakness	CWE-362 (race condition), CWE-364 (signal handler race condition)
Affected	Portable OpenSSH 8.5p1 to 9.7p1 inclusive. Versions before 4.4p1 are also vulnerable unless patched for CVE-2006-5051 and CVE-2008-4109. OpenBSD is not affected.
Fixed in	OpenSSH 9.8p1 (released 1 July 2024), or a vendor backport
Exploitation	Demonstrated on glibc-based Linux. OpenSSH states about 6 to 8 hours of continuous connections on average on 32-bit Linux with ASLR.
Privileges gained	root (sshd's pre-authentication code runs as root in the affected versions)
Interim mitigation	<code>LoginGraceTime 0</code> (removes the race, but exposes sshd to connection-slot denial of service)
CISA KEV	Not confirmed as listed in the sources checked. Check the KEV catalog yourself before using it for prioritization.

Recommendation. Treat every internet-reachable sshd on an affected version as a priority patch. Internal-only hosts follow within the normal high-severity SLA. Where patching must wait, apply the interim mitigation and restrict who can reach port 22.

2. How the vulnerability works

Understanding the mechanism explains why patching, and not authentication hardening, is the fix.

- **Pre-authentication.** The flaw is reachable before login. Key-only authentication, strong passwords and MFA do not help, because the attacker never gets that far.

- **The timer.** Each unauthenticated connection has `LoginGraceTime` seconds to log in. When it expires, `sshd`'s `SIGALRM` handler runs and, in affected versions, calls functions such as `syslog()` that use `malloc()/free()` internally.
- **The race.** If the signal arrives while the main code is inside `malloc()` or `free()`, the heap can be left in an inconsistent state, and the handler's own allocation then corrupts memory.
- **The history.** CVE-2006-5051 fixed this class of bug in 2006. A later change (introduced October 2020, first shipped in 8.5p1) accidentally removed the protection. Versions 4.4p1 to 8.4p1 are not vulnerable for this reason.
- **Why it is hard, but not safe to ignore.** The attacker must win a narrow timing window and defeat ASLR, so it takes thousands of connection attempts. Qualys and others showed this is practical against glibc-based Linux. Attackers can automate it and run it for hours against exposed hosts.

Related issue

A separate but related race condition, CVE-2024-6409, affects a different code path in some builds (notably the OpenSSH version shipped with RHEL 9). Check your vendor's advisory to confirm the fixed package covers both.

3. Affected and not affected systems

System	Status	Notes
Portable OpenSSH 8.5p1 to 9.7p1	Vulnerable	Upgrade to 9.8p1 or later
Portable OpenSSH 9.8p1 and later	Fixed	9.9p1 adds <code>PerSourcePenalties</code> tuning and random jitter on the grace timer
Portable OpenSSH 4.4p1 to 8.4p1	Not vulnerable	The 2006 fix was still in place
Portable OpenSSH before 4.4p1	Vulnerable unless patched	Also affected by older CVEs; upgrade regardless
OpenBSD sshd	Not affected	Uses a safe mechanism since 2001
Ubuntu 24.04 / 22.04	Fixed by backport	Package 1:9.6p1-3ubuntu13.3 (noble) and 1:8.9p1-3ubuntu0.10 (jammy) or later (USN-6859-1). Ubuntu 20.04 and earlier are listed as not affected.
Debian 12 (bookworm)	Fixed by backport	Tracker lists 1:9.2p1-2+deb12u10 as the current fixed version. Debian 11 (bullseye) is not affected.
RHEL, Rocky, Alma, Oracle Linux	Version dependent	See Red Hat's CVE page and RHSA-2024:4312 and related advisories. Confirm per major release, since RHEL 8 ships an older OpenSSH than RHEL 9.
Containers, AMIs, VM templates	Check each image	The vulnerable sshd may live in a base image that never gets a host update
Network appliances, IoT, OT devices	Vendor dependent	Many embed OpenSSH. Look up each vendor's PSIRT advisory; you cannot patch these yourself

Package versions above come from the Ubuntu and Debian security trackers at the time of writing. Trackers update as new revisions ship, so any version at or above the listed one is acceptable.

4. Risk and prioritization

Factor	Higher priority	Lower priority
Exposure	sshd reachable from the internet or an untrusted network	Reachable only from a management VLAN or bastion
Version	8.5p1 to 9.7p1 with no vendor backport	Vendor package already contains the fix
Platform	glibc-based Linux (demonstrated exploit path)	musl or other libc (not shown to be exploitable, still patch)
Asset value	Jump hosts, build servers, database and hypervisor hosts	Short-lived lab machines
Monitoring	No log review; many sshd connections allowed	Rate limiting and alerting on repeated timeouts in place

Suggested timelines: internet-facing hosts within 72 hours where possible, internal hosts within your standard high-severity window (often 14 to 30 days). Adjust to your own policy.

5. Step 1: find and confirm vulnerable systems

On a host you can log in to

```
ssh -V
sshd -T | grep -i loggingracetime
# Debian / Ubuntu
dpkg -l openssh-server | tail -1
apt changelog openssh-server | grep -i -m3 CVE-2024-6387
# RHEL / Rocky / Alma / Fedora / Amazon Linux
rpm -q openssh-server
rpm -q --changelog openssh-server | grep -i -m3 CVE-2024-6387
```

If the changelog mentions CVE-2024-6387, the installed package contains the fix even if the version reads 8.9p1 or 9.2p1. If it does not, treat the host as vulnerable when the upstream version falls in 8.5p1 to 9.7p1.

Across the network (authorized scans only)

```
# Banner grab for one host
nc -w 3 HOST 22 | head -1
# Version detection across a range, saved for the report
nmap -sV -p 22 --open -oA ssh-inventory 10.0.0.0/24
```

Limitation. A remote banner shows the upstream version (for example SSH-2.0-OpenSSH_9.6p1 Ubuntu-3ubuntu13.3), and some builds hide the distro suffix. A banner alone can therefore give false positives on backported systems. Confirm on the host with the changelog check, or use an authenticated scan.

Tools that can detect it

Tool	How it helps	Type
Nessus / Tenable	Plugins for CVE-2024-6387; authenticated scans read the package version and changelog	Commercial
Qualys VMDB	Detection with authenticated checks; discoverer of the flaw	Commercial
Rapid7 InsightVM	Vulnerability checks with distro-aware logic	Commercial
OpenVAS / Greenbone	Network vulnerability tests including version checks	Open source
Nmap (-sV)	Fast inventory of sshd versions; pair with host checks	Open source
Wazuh	Agent-based vulnerability detection from installed package data	Open source
Trivy / Gype	Scan container images and filesystems for the vulnerable package	Open source
Ansible / osquery	Collect package versions from a whole fleet in one run	Open source

Public proof-of-concept code exists for this CVE. Do not run untrusted PoC code against production systems, and only test in a lab you own or with written authorization.

6. Step 2: choose a remediation route

Route	When to use	Trade-off
A. Vendor package update (preferred)	Any distribution that ships a fixed package	Fastest, supported, keeps package manager state clean
B. Build 9.8p1 or later from source	Vendor has no fix, or you need a newer feature	You own future patching; can conflict with package manager
C. Vendor firmware / appliance update	Network and OT devices with embedded OpenSSH	Depends on vendor timeline; may need a maintenance window
D. Interim mitigation only	Patch cannot happen yet	Reduces exposure but is not a fix (section 7)

Route A: vendor package update

Debian / Ubuntu

```
sudo apt-get update
sudo apt-get install --only-upgrade openssh-server openssh-client openssh-sftp-server
apt changelog openssh-server | grep -i -m3 CVE-2024-6387
```

RHEL / Rocky / AlmaLinux / Oracle Linux / Amazon Linux / Fedora

```
sudo dnf clean all
sudo dnf update openssh openssh-server openssh-clients
rpm -q --changelog openssh-server | grep -i -m3 CVE-2024-6387
```

SUSE / openSUSE

```
sudo zypper refresh
sudo zypper update openssh
```

Alpine

```
apk update && apk upgrade openssh
```

FreeBSD (base system sshd)

```
sudo freebsd-update fetch install
```

Package upgrades normally restart or reload sshd on their own. Confirm the running process is the new one:

```
systemctl status ssh 2>/dev/null || systemctl status sshd
ps -o pid,lstart,cmd -C sshd | head -3
```

Route B: build from source (last resort)

Use this only when no vendor package is available. It bypasses your package manager, so record it as a manual install and remember that a later distro update may overwrite it. Replace 9.9p1 with the current release from openssh.com and verify the download with the project's published signature before building.

```
cd /tmp
curl -fLO https://cdn.openbsd.org/pub/OpenBSD/OpenSSH/portable/openssh-9.9p1.tar.gz
curl -fLO https://cdn.openbsd.org/pub/OpenBSD/OpenSSH/portable/openssh-9.9p1.tar.gz.asc
# verify the signature with the release key published on openssh.com, then:
sudo apt-get install -y build-essential zlib1g-dev libssl-dev libpam0g-dev # Debian/Ubuntu
tar xzf openssh-9.9p1.tar.gz && cd openssh-9.9p1
./configure --prefix=/usr --sysconfdir=/etc/ssh --with-pam --libexecdir=/usr/lib/openssh
make -j"$(nproc)"
sudo cp -a /etc/ssh /etc/ssh.bak-"$(date +%F)"
sudo make install
sudo sshd -t && sudo systemctl restart ssh # service is "sshd" on RHEL family
```

Adjust `--libexecdir` to match your distribution's layout (RHEL family uses `/usr/libexec/openssh`). From 9.8 onward, sshd runs a separate `sshd-session` binary, so a partial install that misses it will stop logins.

7. Interim mitigation when you cannot patch yet

Apply these to reduce risk, then schedule the real patch. They are not a substitute for it.

7.1 Disable the login grace timer

Setting `LoginGraceTime 0` removes the signal handler path that triggers the race. The downside is that unauthenticated connections no longer time out, so an attacker can fill the `MaxStartups` slots and block legitimate logins (denial of service).

```
echo 'LoginGraceTime 0' | sudo tee /etc/ssh/sshd_config.d/99-regresshion.conf
sudo sshd -t && sudo sshd -T | grep -i logingracetime
sudo systemctl reload ssh # "sshd" on RHEL family
```

If `sshd -T` still shows a non-zero value, an earlier setting is winning (sshd uses the first value it reads). Edit the main file or move the Include line.

7.2 Reduce who can reach sshd

- Restrict port 22 to known source ranges or a VPN/bastion host in the firewall or cloud security group.
- Tune `MaxStartups` and, on 9.8+, use `PerSourcePenalties` to throttle repeat offenders.
- Use fail2ban or your firewall's rate limiting against repeated connections from one source.
- Do not rely on changing the port number or on key-only authentication. Neither stops a pre-authentication bug.

```
# ufw example: allow SSH only from one management range
sudo ufw allow from 203.0.113.0/24 to any port 22 proto tcp
sudo ufw deny 22/tcp
# firewallld example
sudo firewall-cmd --permanent --add-rich-rule='rule family="ipv4" \
  source address="203.0.113.0/24" port port="22" protocol="tcp" accept'
sudo firewall-cmd --permanent --remove-service=ssh && sudo firewall-cmd --reload
```

8. Step 3: roll out safely

Before you change anything

- Inventory the hosts, the current version and the owner of each. Export the nmap or scanner results as your baseline.
- Take a snapshot or backup of critical hosts, and copy /etc/ssh off the host.
- **Keep one existing SSH session open** and open a second one to test with before you close the first. If sshd fails to start, the open session is your way back in. Have console or out-of-band access ready for remote hosts.
- Agree a maintenance window. Restarting sshd does not normally drop existing sessions, but a package upgrade can restart related services.

Staged rollout

Stage	Scope	Gate to continue
1. Canary	2 to 3 non-critical hosts per OS family	Validation checklist (section 9) passes; nothing breaks for 24 hours
2. Non-production	Dev, test, staging	Automation, CI/CD and admin access still work
3. Production, internet-facing first	Batches of about 20 percent	Each batch validated before the next starts
4. Remaining and exceptions	Appliances, images, legacy systems	Vendor fixes tracked; compensating controls documented

Automation options

Ansible, Puppet or Chef for fleets; Red Hat Satellite, Ubuntu Landscape or AWS Systems Manager Patch Manager for managed estates; unattended-upgrades or dnf-automatic for ongoing security updates. Example Ansible play, run first against a canary group with `--limit canary`:

```
- hosts: all
  become: true
  serial: "20%"
  tasks:
    - name: Upgrade OpenSSH (Debian family)
      ansible.builtin.apt: { name: openssh-server, state: latest, update_cache: true }
      when: ansible_os_family == "Debian"
    - name: Upgrade OpenSSH (RedHat family)
      ansible.builtin.dnf: { name: openssh-server, state: latest }
      when: ansible_os_family == "RedHat"
    - name: Validate sshd configuration
      ansible.builtin.command: sshd -t
      changed_when: false
    - name: Restart sshd
      ansible.builtin.service:
        name: "{{ 'ssh' if ansible_os_family == 'Debian' else 'sshd' }}"
        state: restarted
```

`state: latest` installs the newest package in your repositories, which also pulls in unrelated fixes. That is normally what you want for a security update, but test it as you would any other change.

9. Step 4: test and validate

9.1 Post-patch validation checklist

#	Check	Command or method	Expected result
1	Fix is present	<code>apt changelog openssh-server grep CVE-2024-6387</code> or <code>rpm -q --changelog openssh-server grep CVE-2024-6387</code>	CVE listed in changelog, or upstream version 9.8p1 or later
2	Config is valid	<code>sudo sshd -t</code>	No output, exit code 0
3	New process running	<code>ps -o pid,lstart,cmd -C sshd</code>	Start time after the upgrade
4	Listening	<code>ss -tlnp grep :22</code>	sshd listening on expected addresses
5	Effective settings	<code>sudo sshd -T grep -Ei 'loggingracetim e maxstartups permitrootlogin password auth'</code>	Matches your baseline (revert any temporary LoginGraceTime 0)
6	Login works	New terminal: <code>ssh user@HOST</code>	Key and MFA logins succeed
7	File transfer	<code>sftp</code> and <code>scp</code> a test file	Transfer completes
8	Jump hosts and tunnels	<code>ssh -J bastion user@HOST</code> , port forwards	Work as before
9	Automation	Ansible ping, CI/CD deploys, git over SSH, backup jobs	All succeed
10	Logs are clean	<code>sudo journalctl -u ssh --since '15 min ago'</code> (unit is sshd on RHEL)	No errors or crash loops
11	Rescan	Re-run the scanner used for the baseline	Finding no longer reported

9.2 What can go wrong

Symptom	Likely cause	Fix
Cannot log in after upgrade	Config option deprecated, or a source build missed <code>sshd-session</code>	Use the open session or console, run <code>sudo sshd -t</code> , fix the reported line
Old algorithms rejected by clients	Newer OpenSSH disables weak algorithms	Update the client; do not re-enable weak algorithms unless unavoidable
Version scan still flags the host	Scanner reads the banner, not the backport	Confirm via changelog and mark as a false positive with evidence
Legitimate logins blocked after tuning	MaxStartups or firewall too strict	Raise the limit or add the source to the allow list

9.3 Rollback

```
# Debian / Ubuntu: list available versions, then pin the previous one
apt-cache policy openssh-server
sudo apt-get install openssh-server=<previous-version>
# RHEL family: undo the last transaction
sudo dnf history list
sudo dnf history undo <transaction-id>
# Source build: restore the config backup and reinstall the distro package
sudo cp -a /etc/ssh.bak-<date>/ /etc/ssh/
```

Rolling back reintroduces the vulnerability, so use it only to restore access, then fix the underlying problem and patch again. Do not test exploitation on production. If you need to prove exploitability, do it in an isolated lab with written authorization.

10. Detection and monitoring

Qualys notes that attempts show up as very large numbers of *"Timeout before authentication"* messages. This is only a signal, since scanners and misconfigured clients also create timeouts, but a sudden spike from one source is worth investigating.

```
# Count timeout events in the last 24 hours
sudo journalctl -u ssh --since "24 hours ago" | grep -c "Timeout before authentication"
# Top source addresses
sudo journalctl -u ssh --since "24 hours ago" | grep "Timeout before authentication" \
  | grep -oE '([0-9]{1,3}\.){3}[0-9]{1,3}' | sort | uniq -c | sort -rn | head
# Classic log file locations
sudo grep -c "Timeout before authentication" /var/log/auth.log          # Debian / Ubuntu
sudo grep -c "Timeout before authentication" /var/log/secure          # RHEL family
```

Send sshd logs to your SIEM and alert on repeated timeouts from one source, unexpected sshd crashes or restarts, and new root sessions. If a host shows signs of exploitation before patching, treat it as compromised: isolate it, preserve evidence, rebuild from a known-good image and rotate any credentials and SSH host keys that lived on it.

11. Reporting template for a client or internal finding

Field	Content
Title	OpenSSH sshd remote unauthenticated code execution (CVE-2024-6387, regreSSHion)
Severity	High (CVSS 3.1 8.1). Raise to Critical for internet-facing hosts with no compensating controls, per your rating policy.
Affected assets	List of hosts, IPs, versions and owners (from the scan or inventory)
Evidence	Scan output, <code>ssh -V</code> and package version, changelog check result, screenshot or log excerpt
Impact	An unauthenticated remote attacker may gain root and full control of the server, then move laterally or access data
Likelihood	Moderate: complex to exploit, but public research and PoC code exist and exposure is common
Recommendation	Upgrade to a fixed package (OpenSSH 9.8p1 or later, or vendor backport). Interim: <code>LoginGraceTime 0</code> and network restriction
Effort	Low per host (one package, a service restart, no reboot in most cases); higher for images and appliances
Verification	Changelog check, <code>sshd -t</code> , functional tests, and a clean rescan
References	Section 12

12. Special cases

- Containers and images.** Patching the host does not fix an sshd inside an image. Rebuild from an updated base image, then redeploy. Check with `trivy image --severity HIGH,CRITICAL IMAGE`. Better still, remove sshd from images that do not need it.
- Cloud instances and templates.** Update the golden AMI or image and replace instances, otherwise autoscaling brings vulnerable ones back.
- Appliances and OT/IoT.** Many embed OpenSSH and cannot be patched by you. Record the vendor advisory, restrict management access to a dedicated network, and apply the interim mitigation if the vendor supports it. This matters most for OT devices with long patch cycles.
- Non-glibc systems.** The public exploit targets glibc-based Linux. Other platforms were not shown to be exploitable, but they may still be affected and should be patched.

13. References

- NVD: CVE-2024-6387, <https://nvd.nist.gov/vuln/detail/CVE-2024-6387>
- OpenSSH 9.8 release notes, <https://www.openssh.com/txt/release-9.8>
- OpenSSH release notes index, <https://www.openssh.org/releasenotes.html>
- Qualys Threat Research Unit advisory, <https://www.qualys.com/regresshion-cve-2024-6387>
- Ubuntu security notice, <https://ubuntu.com/security/CVE-2024-6387> (USN-6859-1)
- Debian security tracker, <https://security-tracker.debian.org/tracker/CVE-2024-6387>
- Red Hat CVE page and RHSA-2024:4312, <https://access.redhat.com/security/cve/cve-2024-6387>

Facts in this report were checked against NVD, the OpenSSH release notes, Qualys and the Ubuntu and Debian trackers on 25 September 2026. Distribution package versions change as vendors publish new revisions, so confirm the current fixed version with your vendor before you finalize a client deliverable.